Documentation Technique

Gestion de commande

Gestion des produits et des bons de commande

PERSON Yanis
Concepteur Développeur d'Applications (CDA)
Greta Bretagne Sud -- 2026 / 2027

Sommaire

1 – Introduction

2 – Présentation du projet

3 – Technologies utilisées

4 – Arborescence du projet

5 – Base de données (fichiers texte)
- 5.1 Produits.txt
- 5.2 Commandes.txt
- 5.3 etat.txt

6 – Description des classes
- 6.1 Package fr.cda.projet
- 6.2 Package fr.cda.models
- 6.3 Packages fournis : fr.cda.ihm et fr.cda.util

7 – Déroulement du projet
- Étape 0 – Mise en place du projet
- Fonctionnalité 1 – Lecture des fichiers texte
  - Extrait – Site.java – méthode initialiserCommande()
  - Extrait – Commande.java – constructeurs (surcharge)
- Fonctionnalité 2 – Afficher tous les produits
  - Extrait – Produit.java – méthode toString()
- Fonctionnalité 3 – Afficher tous les bons de commande
  - Extrait – Site.java – méthode listerToutesCommandes()
- Fonctionnalité 4 – Afficher un bon de commande
  - Extrait – Site.java – méthodes trouverCommande() et listerCommande()
  - Extrait – GUISite.java – bouton Afficher
- Fonctionnalité 5 – Livraison des bons de commande
  - Extrait – Site.java – méthode livraison()
  - Extrait – GUISite.java – bouton Livrer
  - Jeu d'essai avec les fichiers fournis
- Fonctionnalité 6 – Modification d'un bon de commande
  - Extrait – GUISite.java – bouton Modifier
  - Extrait – GUIModifierCommande.java – méthode submit()
- Fonctionnalité 7 – Somme des ventes
  - Extrait – Site.java – méthode calculerVentes()
- Fonctionnalité 8 – Envoi de l'état par email
  - Extrait – EnvoiMail.java – configuration et session
  - Extrait – EnvoiMail.java – construction et envoi du message
  - Extrait – GUISite.java – bouton Email
- Fonctionnalité 9 – Sauvegarde FTP chiffrée AES
- JAVADOC et conventions de codage
- Versioning

8 – Glossaire technique

9 – Améliorations possibles

10 – Conclusion

1 – Introduction

Ce document est la documentation technique du projet Gestion de commande, développée en Java dans le cadre de la première soutenance de la formation CDA du Greta de Vannes. Il s'adresse aux développeurs et aux personnes en charge de la maintenance ou de l'évolution de l'application.

Il décrit les technologies utilisées, le modèle de données manipulé (deux fichiers texte), les classes de chaque package, les algorithmes principaux ainsi que le déroulement du projet, de la première à la dernière fonctionnalité.

Ce projet a été réalisé individuellement par PERSON Yanis.

2 – Présentation du projet

Le programme permet de gérer le stock de produits et les bons de commande d'une application. Le stock contient des produits identifiés par une référence unique. Chaque bon de commande contient les références des produits commandés, avec la quantité souhaitée pour chacun.

Au démarrage, la classe **Projet** crée un objet **Site**, qui charge en mémoire les deux fichiers de données, puis ouvre la fenêtre principale **GUISite**. Toutes les opérations (affichage, livraison, modification, calcul des ventes, envoi par email) sont déclenchées par les boutons de cette fenêtre.

Fonctionnalités demandées par le sujet :

N°	Fonctionnalité
1	Lire les deux fichiers texte (produits et bons de commande)
2	Afficher tous les produits du stock
3	Afficher tous les bons de commande
4	Afficher un bon de commande en particulier
5	Livrer les bons de commande non livrés (mise à jour du stock, motifs de non-livraison)
6	Modifier les quantités d'un bon de commande non livré
7	Afficher la somme en euros des ventes (bons livrés)
8	Envoyer par email l'état des bons de commande livrés

3 – Technologies utilisées

Technologie	Rôle
Java (JDK 26)	Langage de programmation
Maven	Gestion du projet et des dépendances (fichier pom.xml)
IntelliJ IDEA	Environnement de développement
Swing / classe Formulaire	Interface graphique : formulaire fourni dans le package fr.cda.ihm, construit sur Swing
Fichiers texte	Stockage des données : produits et bons de commande
Jakarta Mail	Bibliothèque d'envoi d'email avec pièce jointe (protocole SMTP)
Serveur SMTP Gmail	Serveur d'envoi du courriel (smtp.gmail.com, port 465, connexion SSL)
Git / GitHub	Gestion de versions
JAVADOC	Documentation du code source

4 – Arborescence du projet

Dossier / Fichier	Description
data/	Fichiers de données (à la racine du projet, à côté de src)
data/Produits.txt	Produits du stock
data/Commandes.txt	Lignes des bons de commande
data/etat.txt	État des commandes livrées, généré avant l'envoi par email
src/main/java/fr/cda/projet/	Package principal : Projet, Site, GUISite, GUIModifierCommande, EnvoiMail
src/main/java/fr/cda/models/	Package des classes métier : Produit, Commande
src/main/java/fr/cda/ihm/	Package fourni : Formulaire
src/main/java/fr/cda/util/	Package fourni : Terminal
docs/	JAVADOC générée (index.html)
pom.xml	Configuration Maven et dépendances

5 – Base de données (fichiers texte)

L'application n'utilise pas de système de gestion de base de données : les données sont stockées dans deux fichiers texte du dossier **data/**.

5.1 Produits.txt

Chaque ligne du fichier est un produit du stock.

Champ	Type Java	Description
Référence	String	Identifiant unique du produit (ex : LIVRE-1).
Nom	String	Nom du produit
Prix	double	Prix en euros
Quantité	int	Quantité disponible en stock

```
LIVRE-1;Les Miserables de Victor Hugo;8.50;6
LIVRE-2;La Peste de Albert Camus;8.5;4
VETEMENT-1;Pantalon bleu taille 44 homme;12.0;5
VETEMENT-2;Pantalon jean femme 48;23.0;4
VETEMENT-3;T-shirt rose femme 52;30.0;6
LIVRE-3;Le Père Goriot de Honoré de Balzac;11.0;5
LIVRE-4;L'étranger de Albert Camus;11.0;5
LIVRE-5;Voyage au bout de la nuit de Louis-ferdinand Céline;11.0;3
BD-1;Tintin au Tibet;11.5;2
BD-2;Spirou et Fantasio;11.5;2
```

5.2 Commandes.txt

Chaque ligne du fichier est un produit d'un bon de commande.

Champ	Type Java	Description
Numéro	int	Numéro unique du bon de commande
Date	String	Date de création au format JJ/MM/AAAA
Client	String	Nom du client
Référence = quantité	String	Référence du produit commandé et quantité demandée (ex : LIVRE-1=8)

```
1;21/09/2022;Macron Brigitte;LIVRE-1=1
1;21/09/2022;Macron Brigitte;LIVRE-2=2
1;21/09/2022;Macron Brigitte;VETEMENT-1=3
2;22/09/2022;Chirac Bernadette;VETEMENT-2=3
2;22/09/2022;Chirac Bernadette;VETEMENT-3=7
2;22/09/2022;Chirac Bernadette;LIVRE-3=7
3;23/09/2022;Martin Jacques;LIVRE-1=8
4;23/09/2022;Bruni Carla;LIVRE-1=9
```

5.3 etat.txt

Ce fichier n'est pas fourni : il est généré par **Site.ecrireEtat()**. Il contient le détail des commandes livrées (numéro, nom, quantité et prix de chaque produit) suivi de la somme en euros.

6 – Description des classes

Les classes sont réparties en packages commençant tous par **fr.cda** Les attributs de toutes les classes sont privés.

6.1 Package fr.cda.projet

Classe Projet

Point d'entrée du programme.

Méthode	Description
main(String[] a_args)	Affiche un message dans la console, crée le **Site** (chargement des fichiers) puis la fenêtre **GUISite**

Classe Site

Classe centrale : elle contient le stock (**ArrayList<Produit>**) et les bons de commande (**ArrayList<Commande>**).

Méthode	Description
Site()	Crée les deux listes puis charge les fichiers **data/Produits.txt** et **data/Commandes.txt**
initialiserStock(nomFichier)	(privée) Lit le fichier des produits et remplit le stock ; ignore les lignes vides et signale les lignes mal formées
initialiserCommande(nomFichier)	(privée) Lit le fichier des commandes ; crée la commande si elle n'existe pas, puis lui ajoute la ligne **REF=quantité**
trouverCommande(int numero)	Retourne la commande de ce numéro, ou **null** si elle n'existe pas
trouverProduit(String reference)	Retourne le produit du stock de cette référence, ou **null** si elle est inconnue
listerTousProduits()	Retourne le texte de tous les produits du stock
listerToutesCommandes()	Retourne le texte de tous les bons de commande
listerCommande(int numero)	Retourne le texte d'un bon de commande, ou un message si le numéro n'existe pas
livraison()	Livre les commandes non livrées dans l'ordre ; retourne la liste des commandes qui n'ont pas pu l'être
calculerVentes()	Retourne le détail des commandes livrées et la somme des ventes en euros
ecrireEtat()	Écrit le résultat de **calculerVentes()** dans le fichier **data/etat.txt**

Classe GUISite

Fenêtre principale, construite avec la classe **Formulaire** fournie. Elle implémente **FormulaireInt** : la méthode **submit** est appelée à chaque clic sur un bouton et exécute l'action correspondante.

Bouton (nom du submit)	Action
Tous le stock (AFF_STOCK)	Affiche **site.listerTousProduits()** dans la zone de résultats
Toutes les commandes (AFF_COMMANDES)	Affiche **site.listerToutesCommandes()**
Afficher (AFF_COMMANDE)	Lit le numéro saisi et affiche **site.listerCommande(numero)** message si la saisie n'est pas un entier
Livrer (LIVRER)	Appelle **site.livraison()** et affiche les commandes non livrées avec leurs motifs
Modifier (MODIFIER)	Retrouve la commande saisie ; ouvre **GUIModifierCommande** si elle existe et n'est pas livrée, sinon affiche un message
Calculer ventes (CALCULER)	Affiche le résultat de **site.calculerVentes()**
Email (EMAIL)	Écrit **data/etat.txt**, appelle **EnvoiMail.envoyer()** et affiche une fenêtre modale de réussite ou d'échec

Classe GUIModifierCommande

Fenêtre de modification des quantités d'une commande non livrée : une zone de saisie par référence, préremplie avec la quantité actuelle, et un bouton « Valider ».

Méthode	Description
GUIModifierCommande(Commande)	Crée le formulaire : une zone de saisie par ligne de la commande, nommée par la référence du produit
submit(Formulaire, String)	Au clic sur Valider : relit et vérifie les quantités, met à jour la commande, ferme la fenêtre

Classe EnvoiMail

Envoi par email du fichier d'état en pièce jointe. Le destinataire, le sujet et le corps du message sont des constantes. Le mot de passe n'est jamais écrit dans le code : il est lu dans la variable d'environnement **MAIL_PASSWORD**.

Méthode	Description
envoyer(String fichier)	Méthode statique : construit et envoie le message ; retourne `true` si l'envoi a réussi, `false` sinon

6.2 Package fr.cda.models

Classe Produit

Un produit du stock : référence (unique), nom, prix et quantité en stock.

Élément	Description
Produit(reference, nom, prix, quantite)	Constructeur
getReference(), getNom(), getPrix(), getQuantite()	Accesseurs
setReference(), setNom(), setPrix(), setQuantite()	Accesseurs, **setQuantite** sert à diminuer le stock lors de la livraison
toString()	Texte aligné en colonnes : référence, nom, prix et quantité

Classe Commande

Un bon de commande : numéro (unique), date (JJ/MM/AAAA), client, liste des lignes **REF=quantité**, indicateur de livraison et motif de non-livraison.

Élément	Description
Commande(numero, date, client, references)	Constructeur complet ; la commande est créée non livrée
Commande(numero, date, client)	Constructeur surchargé : crée une liste de références vide.
getNumero(), getDate(), getClient(), getReferences()	Accesseurs
setReferences(ArrayList<String>)	Remplace les lignes de la commande (modification des quantités)
isLivre() / setLivre(boolean)	Lit ou modifie l'état de livraison
getRaisonPasLivre() / setRaisonPasLivre(String)	Lit ou modifie le texte des produits manquants
toString()	Texte de la commande : numéro, date, client, références, ligne de tirets

6.3 Packages fournis : fr.cda.ihm et fr.cda.util

Ces classes sont fournies avec le sujet et n'ont pas été modifiées. Voici celles utilisées par l'application :

Classe	Rôle
Formulaire (ihm)	Construit une fenêtre : **addLabel, addButton, addText, addZoneText, afficher, setValeurChamp, getValeurChamp, fermer**
FormulaireInt (ihm)	Interface à implémenter pour recevoir les clics : méthode submit(Formulaire, String)
Terminal (util)	**lireFichierTexte** lit un fichier dans un tableau de lignes **ecrireFichier** écrit un fichier

7 – Déroulement du projet

Le projet a été réalisé progressivement, une fonctionnalité après l'autre, avec une version qui fonctionne et un commit Git à la fin de chaque étape.

Étape 0 – Mise en place du projet

Objectif :
Préparer l'environnement de développement et la structure de l'application.

Fichiers concernés :
pom.xml, data/Produits.txt, data/Commandes.txt

Fonctionnalité 1 – Lecture des fichiers texte

Objectif :
Charger en mémoire les produits et les bons de commande au démarrage.

Fichiers concernés :
Site.java, Commande.java, Produit.java

Extrait – Site.java – méthode initialiserCommande()

```
private void initialiserCommande(String nomFichier)
{
    String[] lignes = Terminal.lireFichierTexte(nomFichier);
    if (lignes == null) {
        System.out.println("Le fichier n'a pas pu être lu");
        return;
    }

    for (String ligne : lignes) {
        if (ligne.trim().isEmpty()) {
            continue;
        }
        String[] champs = ligne.split(";");

        try {
            int numero = Integer.parseInt(champs[0].trim());
            String date = champs[1].trim();
            String client = champs[2].trim();

            // La commande existe-t-elle déjà ?
            Commande c = trouverCommande(numero);
            if (c == null) {
                c = new Commande(numero, date, client);
                commandes.add(c);
            }
            c.getReferences().add(champs[3].trim());

        } catch (NumberFormatException e) {
            System.out.println("Ligne mal écrite (nombre invalide) : " + ligne);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Ligne mal écrite (champs manquants) : " + ligne);
        }
    }
}
```

Extrait – Commande.java – constructeurs (surcharge)

```
public Commande(int numero, String date, String client,
                ArrayList<String> references) {
    this.numero = numero;
    this.date = date;
    this.client = client;
    this.references = references;
    this.livre = false;
    this.raisonPasLivre = "";
}

public Commande(int numero, String date, String client) {
    this(numero, date, client, new ArrayList<String>());
}
```

Fonctionnalité 2 – Afficher tous les produits

Objectif :
Afficher le stock dans la zone de résultats.

Fichiers concernés :
Site.java, Produit.java, GUISite.java

Extrait – Produit.java – méthode toString()

```
@Override
public String toString()
{
    return String.format("%-15s %-50s %3.2f    %3d", reference, nom, prix, quantite);
}
```

Fonctionnalité 3 – Afficher tous les bons de commande

Objectif :
Afficher tous les bons de commande avec leurs références.

Fichiers concernés :
Commande.java, Site.java, GUISite.java

Extrait – Site.java – méthode listerToutesCommandes()

```
public String listerToutesCommandes()
{
    String res = "";
    for (Commande commande : commandes) {
        res = res + commande.toString();
    }
    return res;
}
```

Fonctionnalité 4 – Afficher un bon de commande

Objectif :
Afficher le bon dont le numéro est saisi, en testant que ce numéro existe.

Fichiers concernés :
Site.java, GUISite.java

Extrait – Site.java – méthodes trouverCommande() et listerCommande()

```
public Commande trouverCommande(int numero) {
    for (Commande c : commandes) {
        if (c.getNumero() == numero) {
            return c;
        }
    }
    return null;
}

public String listerCommande(int numero) {
    Commande commande = trouverCommande(numero);

    if (commande == null) {
        return "Commande " + numero + " inexistante";
    }
    return commande.toString();
}
```

Extrait – GUISite.java – bouton Afficher

```
if (nomSubmit.equals("AFF_COMMANDE")) {
    try {
        String numStr = form.getValeurChamp("NUM_COMMANDE");
        int num = Integer.parseInt(numStr.trim());
        String res = site.listerCommande(num);
        form.setValeurChamp("RESULTATS", res);
    } catch (NumberFormatException e) {
        form.setValeurChamp("RESULTATS",
                "Numéro de commande invalide : saisissez un nombre entier");
    }
}
```

Fonctionnalité 5 – Livraison des bons de commande

Objectif :
Livrer les bons non livrés : mettre à jour le stock pour les commandes complètes, marquer les autres comme non livrées avec le détail des manques.

Fichiers concernés :
Commande.java, Site.java, GUISite.java

Extrait – Site.java – méthode livraison()

```
public ArrayList<Commande> livraison() {
    ArrayList<Commande> nonLivrees = new ArrayList<>();

    for (Commande c : commandes) {
        if (c.isLivre()) {
            continue;
        }
        String manque = "";

        // Phase 1 : vérification, sans toucher au stock
        for (String ligne : c.getReferences()) {
            String[] separateur = ligne.split("=");
            String reference = separateur[0];
            int quantiteDemande = Integer.parseInt(separateur[1]);
            Produit p = trouverProduit(reference);

            if (p == null) {
                manque = manque + "Produit inconnu " + reference + "\n";
            } else if (quantiteDemande > p.getQuantite()) {
                manque = manque + "Il manque "
                        + (quantiteDemande - p.getQuantite())
                        + " " + reference + "\n";
            }
        }

        // Phase 2 : si rien ne manque, on diminue le stock et on livre
        if (manque.equals("")) {
            for (String ligne : c.getReferences()) {
                String[] separateur = ligne.split("=");
                Produit p = trouverProduit(separateur[0]);
                int quantiteDemande = Integer.parseInt(separateur[1]);
                p.setQuantite(p.getQuantite() - quantiteDemande);
            }
            c.setLivre(true);
        } else {
            c.setRaisonPasLivre(manque);
            nonLivrees.add(c);
        }
    }
    return nonLivrees;
}
```

Extrait – GUISite.java – bouton Livrer

```
if (nomSubmit.equals("LIVRER")) {
    ArrayList<Commande> nonLivrees = site.livraison();

    String res = "Les commandes non livrées :\n====================\n";
    for (Commande c : nonLivrees) {
        res = res + c.toString() + c.getRaisonPasLivre() + "\n";
    }
    form.setValeurChamp("RESULTATS", res);
}
```

Jeu d'essai avec les fichiers fournis

Commande	Lignes demandées	Résultat
1	LIVRE-1 ×1, LIVRE-2 ×2, VETEMENT-1 ×3	Livrée
2	VETEMENT-2 ×3, VETEMENT-3 ×7, LIVRE-3 ×7	Non livrée : « Il manque 1 VETEMENT-3 » et « Il manque 2 LIVRE-3 »
3	LIVRE-1 ×8 (il en reste 5 après la commande 1)	Non livrée : « Il manque 3 LIVRE-1 »
4	LIVRE-1 ×9	Non livrée : « Il manque 4 LIVRE-1 »

Stock après livraison : LIVRE-1 passe de 6 à 5, LIVRE-2 de 4 à 2 et VETEMENT-1 de 5 à 2 ; les autres produits sont inchangés. Un second clic sur « Livrer » ne modifie plus le stock : les commandes déjà livrées sont ignorées.

Fonctionnalité 6 – Modification d'un bon de commande

Objectif :
Modifier les quantités des références d'un bon de commande non livré.

Fichiers concernés :
GUISite.java, GUIModifierCommande.java, Commande.java

Extrait – GUISite.java – bouton Modifier

```
if (nomSubmit.equals("MODIFIER")) {
    try {
        int num = Integer.parseInt(form.getValeurChamp("NUM_COMMANDE").trim());
        Commande c = site.trouverCommande(num);

        if (c == null) {
            form.setValeurChamp("RESULTATS", "Commande inexistante");
        } else if (c.isLivre()) {
            form.setValeurChamp("RESULTATS", "Commande " + c.getNumero()
                    + " déjà livrée : modification impossible");
        } else {
            new GUIModifierCommande(c);
        }
    } catch (NumberFormatException e) {
        form.setValeurChamp("RESULTATS",
                "Numéro de commande invalide : saisissez un nombre entier");
    }
}
```

Extrait – GUIModifierCommande.java – méthode submit()

```
public void submit(Formulaire form, String nomSubmit) {
    if (nomSubmit.equals("VALIDER")) {
        ArrayList<String> references = new ArrayList<>();

        for (String ligne : commande.getReferences()) {
            String[] separateur = ligne.split("=");
            String ref = separateur[0];
            String quantite = form.getValeurChamp(ref).trim();

            // Vérification de la saisie avant d'enregistrer quoi que ce soit
            try {
```

```
                if (Integer.parseInt(quantite) < 0) {
                    throw new NumberFormatException();
                }
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(null, "Quantité invalide pour "
                        + ref + " : saisissez un entier positif ou nul");
                return;
            }
            references.add(ref + "=" + quantite);
        }
        commande.setReferences(references);
        form.fermer();
    }
}
```

Fonctionnalité 7 – Somme des ventes

Objectif :

Afficher la somme en euros des ventes des bons de commande livrés.

Fichiers concernés :

Site.java, GUISite.java

Extrait – Site.java – méthode calculerVentes()

```
public String calculerVentes() {
    double total = 0;
    String res = "";

    for (Commande c : commandes) {
        if (c.isLivre()) {
            res = res + "COMMANDE : " + c.getNumero() + "\n";

            for (String ligne : c.getReferences()) {
                String[] separateur = ligne.split("=");
                int quantite = Integer.parseInt(separateur[1]);
                Produit p = trouverProduit(separateur[0]);

                res = res + p.getNom() + "    " + quantite
                        + "    " + p.getPrix() + "\n";
                total = total + p.getPrix() * quantite;
            }
            res = res + "\n";
        }
    }
    res = res + "====================\n";
    res = res + "SOMME : " + total + " euros";
    return res;
}
```

Fonctionnalité 8 – Envoi de l'état par email

Objectif :

Envoyer par email un fichier texte décrivant les bons de commande livrés, avec une fenêtre modale indiquant la réussite ou l'échec.

Fichiers concernés :

EnvoiMail.java, Site.java, GUISite.java, pom.xml

Réglages de connexion :

Propriété	Valeur	Rôle
mail.smtp.host	smtp.gmail.com	Adresse du serveur d'envoi
mail.smtp.port	465	Port SMTP avec chiffrement SSL
mail.smtp.auth	true	Le serveur exige une authentification
mail.smtp.ssl.enable	true	Chiffre la connexion

Extrait – EnvoiMail.java – configuration et session

```
public static boolean envoyer(String fichier) {
    // Sécurité : le mot de passe est lu dans une variable d'environnement
    final String motDePasse = System.getenv("MAIL_PASSWORD");
    if (motDePasse == null) {
        System.out.println("Variable MAIL PASSWORD absente");
        return false;
    }

    Properties prop = new Properties();
    prop.put("mail.smtp.host", "smtp.gmail.com");
    prop.put("mail.smtp.port", "465");
    prop.put("mail.smtp.auth", "true");
    prop.put("mail.smtp.ssl.enable", "true");

    Session session = Session.getInstance(prop, new Authenticator() {
        protected PasswordAuthentication getPasswordAuthentication() {
            return new PasswordAuthentication(EXPEDITEUR, motDePasse);
        }
    });
    // ... suite : construction et envoi du message
```

Extrait – EnvoiMail.java – construction et envoi du message

```
try {
    MimeBodyPart texte = new MimeBodyPart();
    texte.setText(CORPS);

    MimeBodyPart piece = new MimeBodyPart();
    piece.attachFile(fichier);

    Multipart contenu = new MimeMultipart();
    contenu.addBodyPart(texte);
    contenu.addBodyPart(piece);

    Message message = new MimeMessage(session);
    message.setFrom(new InternetAddress(EXPEDITEUR));
    message.setRecipients(Message.RecipientType.TO,
            InternetAddress.parse(DESTINATAIRE));
    message.setSubject(SUJET);
    message.setContent(contenu);

    Transport.send(message);
    return true;

} catch (Exception e) {
    System.out.println("Erreur d'envoi : " + e.getMessage());
    return false;
}
```

Extrait – GUISite.java – bouton Email

```
if (nomSubmit.equals("EMAIL")) {
    site.ecrireEtat();
    boolean ok = EnvoiMail.envoyer("data/etat.txt");

    if (ok) {
        JOptionPane.showMessageDialog(null, "Email envoyé avec succès");
    } else {
        JOptionPane.showMessageDialog(null, "Échec de l'envoi de l'email");
    }
}
```

Fonctionnalité 9 – Sauvegarde FTP chiffrée AES

Objectif :
Envoyer une sauvegarde chiffrée des fichiers de données par FTP.

État :
Non réalisée. Cette fonctionnalité était facultative et n'avait pas été étudiée en cours.

JAVADOC et conventions de codage

Les classes, les constructeurs et les méthodes sont documentés par des commentaires **/** ... */** placés juste au-dessus de chaque élément, avec les balises **@param** et **@return**.
La documentation est générée avec Tools > Generate JavaDoc dans IntelliJ dans le dossier **docs/.**

Versioning

Le code est versionné avec Git et publié sur GitHub. Un commit est réalisé à la fin de chaque fonctionnalité fonctionnelle (lecture des fichiers, livraison, modification, ventes, email, JAVADOC).

8 – Glossaire technique

Terme	Définition
Maven	Outil de gestion de projet Java : téléchargement des dépendances dans le fichier pom.xml
IHM	Interface Graphique
JAVADOC	Documentation générée automatiquement à partir des commentaires.
SMTP	Simple Mail Transfer Protocol : protocole standard d'envoi de courriels
MIME / MimeMultipart	Format des messages électroniques composés de plusieurs parties (texte, pièce jointe)
Mot de passe d'application	Mot de passe de 16 caractères généré par Google pour une application
Variable d'environnement	Valeur fournie au programme par le système au lancement permet de ne pas écrire une information sensible dans le code

9 – Améliorations possibles

Les points suivants n'ont pas été réalisés dans ce projet mais pourraient faire l'objet d'évolutions futures :

- **Sauvegarde FTP chiffrée AES (fonctionnalité 9)**
- **Sauvegarde des modifications**
- **Exceptions personnalisées**
- **Configuration de l'email** : externaliser le destinataire, le sujet et le serveur dans un fichier **.properties.**

10 – Conclusion

L'application répond aux huit fonctionnalités obligatoires du sujet : lecture des fichiers, affichage du stock et des bons de commande, livraison avec mise à jour du stock et motifs de non-livraison, modification des quantités, calcul des ventes et envoi de l'état par email.

Document rédigé dans le cadre du Projet de la première soutenance -- Greta Vannes -- Année 2026-2027

PERSON Yanis